

Лабораторная работа № 3

Тема: Проектирование базы данных Internet Shop

Цель работы: Переработать существующую структуру базы shop_db: выделить категорию товара в отдельную сущность, создать таблицу categories, связать её с products через внешний ключ и перенести существующие данные без потери товаров.

1. Посмотрите текущую структуру products

Выполните:

```
SELECT *
```

```
FROM products
```

```
ORDER BY id;
```

Предположим, таблица содержит:

```
id | name      | price | category | in_stock
```

```
-----
```

```
1 | Keyboard  | 15000 | Accessories | true
```

```
2 | Mouse     | 8000  | Accessories | true
```

```
3 | Monitor   | 85000 | Monitors    | false
```

```
4 | SSD 2TB   | 65000 | Components  | true
```

Обратите внимание: значение Accessories повторяется.

2. Посмотрите уникальные категории

```
SELECT DISTINCT category
```

```
FROM products
```

```
ORDER BY category;
```

Пример:

Accessories

Components

Monitors

3. Создайте таблицу categories

```
CREATE TABLE categories (
```

```
  id INTEGER
```

```
    GENERATED ALWAYS AS IDENTITY
```

```
    PRIMARY KEY,
```

```
  name VARCHAR(100)
```

```
    NOT NULL
```

```
    UNIQUE
```

```
);
```

4. Проверьте структуру

В pgAdmin откройте:

shop_db

→ Schemas
→ public
→ Tables
→ categories
Проверьте:
id
name

5. Перенесите существующие категории

Выполните:

```
INSERT INTO categories (name)
SELECT DISTINCT category
FROM products
ORDER BY category;
```

6. Проверьте категории

```
SELECT *
FROM categories
ORDER BY id;
```

Например:

1 | Accessories

2 | Components

3 | Monitors

Фактические id могут отличаться.

7. Проверьте UNIQUE

Попробуйте:

```
INSERT INTO categories (name)
VALUES ('Accessories');
```

PostgreSQL должен отклонить повторяющееся значение.

После проверки никакую дополнительную строку удалять не требуется: ошибочная вставка не выполняется.

8. Добавьте category_id в products

Сначала создадим столбец без NOT NULL, потому что существующие товары пока не имеют значения category_id.

```
ALTER TABLE products
ADD COLUMN category_id INTEGER;
```

9. Посмотрите промежуточное состояние

```
SELECT
  id,
  name,
  category,
```

```
category_id
FROM products
ORDER BY id;
Сейчас:
category_id = NULL
у существующих товаров.
```

10. Заполните category_id

Используйте существующее текстовое название категории:

```
UPDATE products AS p
SET category_id = c.id
FROM categories AS c
WHERE p.category = c.name;
```

Здесь:

p.category

сравнивается с:

c.name

а найденный:

c.id

записывается в:

p.category_id

11. Проверьте результат

```
SELECT
  id,
  name,
  category,
  category_id
FROM products
ORDER BY id;
```

Пример:

```
1 | Keyboard | Accessories | 1
2 | Mouse   | Accessories | 1
3 | Monitor  | Monitors    | 3
4 | SSD 2TB  | Components  | 2
```

12. Проверьте отсутствие NULL

```
SELECT *
FROM products
WHERE category_id IS NULL;
```

Результат должен содержать:

0 строк

Если строки есть, **не переходите к следующему шагу**, пока не выясните причину.

13. Сделайте category_id обязательным

```
ALTER TABLE products  
ALTER COLUMN category_id  
SET NOT NULL;
```

Теперь каждый товар обязан иметь категорию.

14. Создайте внешний ключ

```
ALTER TABLE products  
ADD CONSTRAINT products_category_id_fkey  
FOREIGN KEY (category_id)  
REFERENCES categories(id);
```

15. Проверьте FOREIGN KEY

Получите максимальный существующий id категории:

```
SELECT MAX(id)  
FROM categories;
```

Затем попробуйте создать товар с явно несуществующим category_id, например:

```
INSERT INTO products  
(  
    name,  
    price,  
    category,  
    category_id,  
    in_stock  
)  
VALUES  
(  
    'Test Product',  
    1000,  
    'Unknown',  
    999999,  
    true  
);
```

PostgreSQL должен отклонить запрос из-за нарушения внешнего ключа.

Внешний ключ требует, чтобы значение ссылочного столбца соответствовало существующей строке в таблице, на которую он ссылается.

16. Сравните две модели

Сейчас products временно содержит и старое поле: category

и новое:

category_id

Это промежуточное состояние, необходимое для безопасного переноса данных.

17. Проверьте данные через JOIN

```
SELECT
```

```
  p.id,
```

```
  p.name,
```

```
  p.price,
```

```
  c.name AS category,
```

```
  p.in_stock
```

```
FROM products AS p
```

```
JOIN categories AS c
```

```
  ON c.id = p.category_id
```

```
ORDER BY p.id;
```

Результат должен содержать те же названия категорий, что и раньше.

18. Удалите старый столбец

Только после успешной проверки выполните:

```
ALTER TABLE products
```

```
DROP COLUMN category;
```

Теперь название категории хранится только в таблице:
categories

19. Проверьте окончательную структуру

```
SELECT *
```

```
FROM products
```

```
ORDER BY id;
```

Таблица должна содержать:

```
id
```

```
name
```

```
price
```

```
in_stock
```

```
category_id
```

20. Проверьте итоговый JOIN

```
SELECT
```

```
  p.id,
```

```
  p.name,
```

```
  p.price,
```

```
  c.name AS category,
```

```
  p.in_stock AS "inStock"
```

```
FROM products AS p
```

```
JOIN categories AS c
  ON c.id = p.category_id
ORDER BY p.id;
```

21. Обновите getAllProducts()

После изменения структуры старый SQL:

```
SELECT category
FROM products;
```

больше работать не будет.

В productService.js используйте:

```
export async function getAllProducts() {
  const result =
    await pool.query(
      `SELECT
        p.id,
        p.name,
        p.price,
        p.category_id AS "categoryId",
        c.name AS category,
        p.in_stock AS "inStock"
      FROM products AS p
      JOIN categories AS c
        ON c.id = p.category_id
      ORDER BY p.id`
    );

  return result.rows;
}
```

22. Обновите findProductById()

```
export async function findProductById(
  id
){
  const result =
    await pool.query(
      `SELECT
        p.id,
        p.name,
        p.price,
        p.category_id AS "categoryId",
        c.name AS category,
        p.in_stock AS "inStock"
      FROM products AS p
      JOIN categories AS c
```

```

        ON c.id = p.category_id
    WHERE p.id = $1`,
    [id]
);

return result.rows[0];
}

```

23. Проверьте REST API

GET /api/products

Пример ответа:

```

[
  {
    "id": 1,
    "name": "Keyboard",
    "price": 15000,
    "categoryId": 1,
    "category": "Accessories",
    "inStock": true
  }
]

```

Обратите внимание: структура PostgreSQL изменилась, но API всё ещё может вернуть удобное для frontend название категории.

24. Проверьте один товар

GET /api/products/1

Ответ должен содержать:

categoryId

category

25. Создайте черновую модель следующих сущностей

Пока не создавайте эти таблицы.

Запишите структуру:

users

id

name

email

orders

id

user_id

status

created_at

order_items

order_id

product_id

quantity

price

Определите:

- какой столбец является первичным ключом;
- какие столбцы будут обязательными;
- какие значения должны быть уникальными;
- какие столбцы являются ссылками на другие таблицы.

Подробно типы связей 1:1, 1:N и M:N разберём в следующей лекции.

Требования к результату

В базе должны присутствовать:

categories

products

categories должна содержать:

id

name

products должна содержать:

id

name

price

category_id

in_stock

Необходимо обеспечить:

- categories.id — PRIMARY KEY;
- categories.name — NOT NULL UNIQUE;
- products.id — PRIMARY KEY;
- products.name — NOT NULL;
- products.price — CHECK (price > 0);
- products.category_id — NOT NULL;
- products.category_id — FOREIGN KEY;
- сохранение всех существующих товаров;
- отсутствие старого текстового столбца category;
- получение названия категории через JOIN;
- работоспособность GET /api/products;
- работоспособность GET /api/products/:id.

